Project 01 — Your first HTTP API

Theory, answers and design decision

APIs: 24 Build-It-Yourself Projects

5 November 2026 • FastAPI 0.162.0, Starlette 1.10.0, Pydantic 2.14.2, uvicorn 0.54.0, Python 3.13.18

How to read this. Every claim about *our* API was checked against the running code (TestClient or a live uvicorn server, 2026-10-30 and 2026-11-05). Claims about HTTP itself cite RFC 9110 (HTTP Semantics). Where something was measured rather than read, the text says so.

Contents

1 HTTP in five parts **2**

2 The layers a request passes through **2**

3 Status codes we return, and why **4**

4 Validation at the boundary **4**

5 Response models: a stable contract (R02) **5**

6 A running process is not a ready service **5**

7 Observing every request without leaking data **5**

8 Edge cases measured on 2026-11-05 **5**

9 The four workbook questions **6**

10 Design decision **7**

11 Cost **7**

12 Reading **7**

1 HTTP in five parts

An HTTP exchange is one request and one response. RFC 9110 separates every message into the same few parts. Our captured exchange shows all of them:

```
> GET /hello?name=Nick HTTP/1.1          <- method + target + version
> Host: 127.0.0.1:8000                   <- headers (metadata)
> Accept: */*
>                                        <- empty line: headers end
< HTTP/1.1 200 OK                        <- status
< content-type: application/json         <- headers describing the body
< content-length: 27
< server-timing: app;dur=2.08
<
{"greeting":"Hello, Nick!"}              <- representation (the body)
```

Part	What it is	In our API
Method	What the client wants to do with the resource. `GET` means "give me a representation"; it is *safe* (no state change) and *idempotent* (repeating it changes nothing) (RFC 9110 §9.2).	Both routes are `GET` only. Anything else on a known path is `405`.
Target	The resource: a *path* plus an optional *query string* after `?`. The router matches on the path only.	`/hello` picks the handler; `name=Nick` becomes the function argument `name`.
Headers	Name/value metadata about the message, not the content itself.	`content-type` and `content-length` come from FastAPI; `date` and `server` from uvicorn; `server-timing` from our middleware.
Status	A three-digit code: 2xx success, 3xx redirect, 4xx client error, 5xx server error (RFC 9110 §15).	We return only 200, 404, 405 and 422, all on purpose (R05).
Representation	The body, plus the headers that say how to read it.	Always JSON, declared by `content-type: application/json` (R03).

1.1 Why the status code is not the whole story

The status tells the client *which kind* of outcome happened; the body tells it *the details*. A `200` with a wrong body is still a broken API. That is why the tests check the status *and* the exact JSON (`response.json() == {...}`), not just the status.

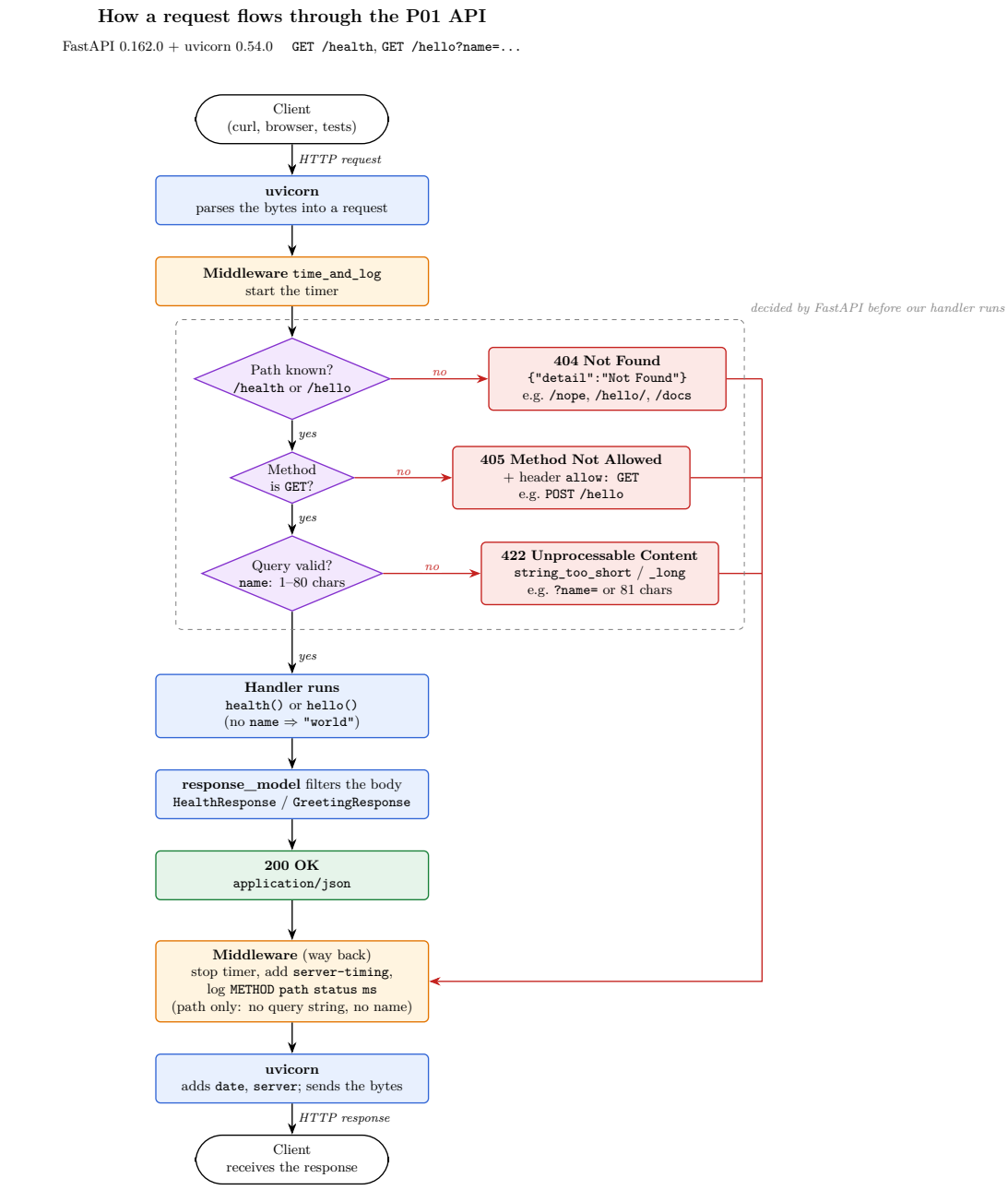
2 The layers a request passes through

Our project is one file, but four libraries handle the request:

1. **uvicorn** (an ASGI server) listens on a TCP port, reads bytes, turns them into a request, and later writes the response bytes. It adds `date` and `server`.

2. **Starlette** (inside FastAPI) runs the middleware and the *router*: it matches the path, then the method.
3. **FastAPI** reads the function signature (`name: str = Query(...)`) and validates the query string *before* calling our function.
4. **Pydantic** does the actual validation, and turns the returned model into JSON through `response_model`.

Our own code is only the middleware `time_and_log` and the two handler functions. Everything else is framework behavior, which is why we *measured* it instead of assuming it (Section 8).



Every answer (200, 404, 405, 422) passes through the middleware, so every response carries `server-timing` and produces exactly one `app.access` log line. Checked live 2026-11-05 with `uvicorn app.main:app --no-access-log --log-config logging.json`.

The order matters: **path, then method, then validation**. So `POST /nope` is a 404 (the path

fails first), `POST /hello` is a 405, and `GET /hello?name=` is a 422. The middleware wraps all of it, so every outcome, including errors, is timed and logged.

3 Status codes we return, and why

Status	Meaning (RFC 9110)	When we send it
`200 OK`	The request succeeded; the body is the result (§15.3.1).	Valid `GET /health` or `GET /hello`.
`404 Not Found`	No current representation for the target (§15.5.5).	Any other path: `/nope`, `/HELLO`, `/hello/`, `/docs`.
`405 Method Not Allowed`	The target exists but does not support this method. The server **MUST** send an `Allow` header listing the methods it does support (§15.5.6).	`POST`, `PUT`, `DELETE`, `PATCH` on either route. We send `allow: GET`.
`422 Unprocessable Content`	The request is well-formed, but its content cannot be processed (§15.5.21). The old name was "Unprocessable Entity".	`name` is empty or longer than 80 characters (R04).

3.1 The status we removed: 307

By default FastAPI answers `GET /hello/` with `307 Temporary Redirect` to `/hello`, with an empty body and no `content-type`. That is a status we never chose, and a body that is not JSON. It breaks R03 and R05. Setting `redirect_slashes=False` makes `/hello/` an ordinary 404. This was our **independently discovered failure**: the test went red (2 failed), and after the fix it went green (22/22).

3.2 404 vs 422: which part of the request was wrong?

404 and 405 are about the *target and method*: the client asked for something that does not exist. 422 is about the *content*: the client reached a real route but sent unacceptable data. Keeping them apart tells the client what to fix.

4 Validation at the boundary

`Query("world", min_length=1, max_length=80)` is a *contract at the boundary*: bad input is rejected before it reaches our code. Inside `hello()`, `name` is guaranteed to be a string of 1–80 characters.

- **Default.** No `name` at all means `"world"` (T01). That is different from `?name=`, which is *present but empty*, so it is rejected (422).
- **Characters, not bytes (measured).** `max_length` counts Unicode characters. 80 emoji (320 bytes in UTF-8) are accepted; 81 emoji are rejected. So R04 bounds characters. If the real concern were memory or log size, a byte limit would be needed as well.
- **The 422 body echoes the input (measured).** The error contains `"input": "aaa...a"`, the rejected value. That is harmless here, but if a field held a password or token, the error

would send it back. Later projects should not echo sensitive fields.

5 Response models: a stable contract (R02)

A client is written against field *names*. If `greeting` became `message`, every client would break, even with a `200`. `response_model=GreetingResponse` pins the shape: undeclared fields are dropped before sending. We measured this: a stray `db_password` key leaked without `response_model` and was dropped with it. A test pins the field names, so a rename now fails a test instead of breaking clients.

6 A running process is not a ready service

`GET /health` returning `{"status":"ok"}` proves only that the process is up and can answer HTTP. It says nothing about dependencies, because this project has none. In later projects (database, queues) there are two different questions:

	Liveness	**Readiness**
Question	Is the process alive?	Can it do real work right now?
Checks	Nothing beyond answering.	Database reachable, migrations applied, ...
If it fails	Restart the process.	Stop sending traffic; do not restart.

Our `/health` is a liveness check. T06 also makes sure it never exposes the environment: with `DATABASE_PASSWORD` set to a canary value, neither the body nor the headers contain it.

7 Observing every request without leaking data

- **Timing.** The middleware measures the time with `time.perf_counter()` (a monotonic clock, safe for durations) and sends it in the `server-timing` header (a W3C standard that browser dev tools display).
- **One log line per request:** `app.access GET /hello 200 2.08ms`.
  It logs `request.url.path`, which leaves out the query string, so the caller's `name` never reaches the log.
- **Two flags are needed on the real server.** uvicorn's own access log prints the full query string, so it is switched off with `--no-access-log`. Our `app.access` logger has no handler unless `--log-config logging.json` gives it one; without that, the lines are silently dropped. Tests could not show this, because pytest captures log records itself.

8 Edge cases measured on 2026-11-05

These were not in the spec. They were measured with FastAPI's TestClient against the current code, to learn what the framework does.

Request	Result	What it teaches
`HEAD /health`	**405**, `allow: GET`	RFC 9110 §9.1 says general-purpose servers **MUST** support `GET` and `HEAD`. FastAPI's `@app.get` does not add `HEAD`. A real gap: noted for later, not fixed in P01.
`OPTIONS /hello`	405, `allow: GET`	No CORS or OPTIONS handling; fine for this scope.
`?name=a&name=b`	200, `Hello, b!`	A repeated parameter: the last value wins, silently.
`?name=%20` (a space)	200, `Hello, !`	`min_length=1` counts a space as a character. "Non-empty" is not "meaningful".
`?name=<script>`	200, echoed	Safe as JSON (`application/json`), but dangerous if a client inserts it into HTML.
`/hello?NAME=Nick`	200, `Hello, world!`	Parameter names are case-sensitive; an unknown one is ignored.
`/health?x=1`	200	Unknown query parameters are ignored, not rejected.
80 / 81 emoji	200 / 422	Length is counted in characters, not bytes.

9 The four workbook questions

1. What information and state cross the contract boundary?

In: a method, a path, and for `/hello` one optional query parameter `name` (1–80 characters). Other headers and parameters are ignored. **Out:** a status (200/404/405/422), a JSON body with fixed fields (`status` or `greeting`, or FastAPI's `detail`), and the headers `content-type`, `content-length`, `server-timing`, and `allow` on 405. **State:** none. The API stores nothing between requests, so every request can be understood on its own. Environment variables are deliberately *not* part of the output (T06).

2. Which invariant would stay true if the framework changed?

The *HTTP contract*, not the code: `GET /hello` with no name returns `{"greeting": "Hello, world!"}`; a name of 81 characters gives 422; an unknown path gives 404 with a JSON body; a wrong method gives 405 with `Allow`; `/health` never contains secrets. These are what `tests/test_api.py` checks through HTTP requests, so the same tests would still judge a rewrite in Flask or Starlette. Only `test_every_route_declares_a_response_model` inspects FastAPI internals; it would have to be rewritten.

3. Which failure could a successful status hide?

- A `200` with the wrong body: a renamed field, or a leaked secret. Tests compare the full JSON, and T06 searches for a canary.
- A `200` from a redirect: an HTTP client follows the `307` silently and reports the final `200`. Our tests use `follow_redirects=False` to see what the server really sent.
- A green test suite that skipped a test: an accidentally indented `def` became a nested function and pytest still said "33 passed". The test count must go up when a test is added.
- A `/health` that says `ok` while a dependency is down (liveness vs readiness, Section 6).
- A request that "works" but leaks the name into uvicorn's access log.

4. What must be measured rather than read in the docs?

Everything in Section 8, plus: the number of routes FastAPI adds by default (6, not 2); the `307` on a trailing slash; whether the log line actually reaches the console on a real server; whether the 422 body echoes input; and **latency**. The `server-timing` values seen so far (0.15–2.08 ms) are single samples. Real percentiles (p50/p95/max) come from `scripts/measure.py` in Milestone 7 and go into EVIDENCE.md. They are not estimated here.

10 Design decision

Chosen: one FastAPI file (`app/main.py`): two Pydantic response models, one middleware, two handlers, run by uvicorn's development server.

Alternative considered: a layered layout (routers, a service layer, dependency injection, separate settings), the structure of Project 04.

	One file (chosen)	**Layered**
Meets R01–R06	Yes (34 tests pass)	Yes, with more code
Request path	Readable top to bottom in one file	Spread across modules
Testability	Whole app via TestClient	Same, plus unit tests per layer
Cost now	82 lines, comments included	Several files for two routes with no logic
Pays off when	Little logic, no state	Real business logic, storage, many routes

Why one file meets this brief. The workbook says: *do not add service layers before you can explain the request path.* With two routes and no state, a service layer would hold nothing. FastAPI was chosen over plain Starlette or `http.server` because it gives validation (R04), JSON media type (R03), response filtering (R02) and correct 404/405 handling (R05) without hand-written code.

What it costs. We depend on framework defaults, and some of them were wrong for us (docs routes, slash redirects, no `HEAD`). Each one needed a measurement and a test. The development server is single-process: fine for learning, not a production deployment.

What would make me change it. A third route with real logic, any storage, or settings that differ between environments. Then routers and a service layer become worth their cost (Project 04).

11 Cost

Nothing billable was run: the API ran only on this laptop. The workbook's worksheet assumes 1 instance × 4 hours × \$0.06 = \$0.24 per practice period. That is a *hypothetical rate*, not a measured invoice. The only measured resources are local: the requests and their `server-timing` durations.

12 Reading

- RFC 9110, *HTTP Semantics*: §9 (methods), §15 (status codes), §8.3 (Content-Type). `https://www.rfc-editor.org/rfc/rfc9110.html`
- FastAPI tutorial: query parameters and string validation; response model. `https://fastapi.tiangolo.com/tutorial/`
- W3C, *Server Timing*. `https://www.w3.org/TR/server-timing/`